

SmartPRS — Installation, Configuration & Troubleshooting

Field engineer's manual · Windows + IIS + MySQL · Linux + MySQL · Smart Biometric Bridge

Field	Detail
Product	SmartPRS by Ametecs — HRM, Payroll & workforce compliance
Platform	Laravel 11 · PHP 8.3 · MySQL 8 / MariaDB
Covers	On-premises install (Windows/IIS and Linux), first-run setup, licence activation, biometric device connection, Smart Biometric Bridge, day-to-day use, troubleshooting
Audience	Ametecs installation engineers and customer IT staff
Version	18 August 2026
Support	ejaz@ametecsindia.com · WhatsApp 9000098877

How to use this manual

Read section 2 before you travel. Most failed installs are missing prerequisites or missing information from the customer, and both are far cheaper to fix from the office.

Section 10 is the troubleshooting section. Every entry in it is a fault that has actually happened on a real install — not a theoretical list. If something goes wrong on site, go there first.

Contents

Section	What it covers	Read it when
1. What you are installing	The pieces, the two ways punches reach SmartPRS, what talks to what	Before your first install
2. Prerequisites	Server, PHP + extensions, database, web server, ports, customer information checklist	Before you travel to site
3. Windows + IIS + MySQL	Full install walkthrough on Windows Server	Windows install
4. Linux + MySQL	Full install walkthrough on Ubuntu/RHEL with nginx + PHP-FPM	Linux install
5. First-run setup	Install wizard, queue worker, scheduler, backups	Every install, both platforms
6. Licensing and activation	Editions, the licence gate, the three credential types, the machine fingerprint, offline activation, renewal, error messages	Every on-prem install
7. Devices and Smart Biometric Bridge	Choosing the ingest path, device setup, API keys, employee mapping	After the app is up
8. Go-live verification	14 checks to run before you hand the system over	Last thing on site
9. Day-to-day usage	What the customer does daily, weekly, monthly	Customer handover / training
10. Troubleshooting	Fault codes T-01 to T-42 — symptom, cause, fix	When something is broken
Appendix A	.env setting reference	Reference
Appendix B	Command cheat sheet, Windows and Linux	Reference
Appendix C	Handover sheet to fill in and leave with the customer	End of install

1. What you are installing, and how it fits together

1.1 The pieces

Piece	What it is	Where it runs
SmartPRS application	The Laravel web application — HR, attendance, payroll, compliance	Customer server (on-prem) or the Ametecs cloud
MySQL database	All customer data	Same server, usually
Web server	IIS on Windows, nginx or Apache on Linux	Same server
Queue worker	Background process that sends email and notifications	Same server — required , see 5.4
Scheduler	Runs the nightly jobs (greetings, absence alerts, renewals)	Same server — required , see 5.5
Biometric device	ZKTeco / eSSL fingerprint or face terminal	Customer premises
Smart Biometric Bridge (SBB)	Windows service that collects punches from any vendor's device and forwards them to SmartPRS	A PC on the customer LAN

1.2 How attendance actually reaches SmartPRS

There are two ingest paths. **Knowing which one a site uses is the single most useful thing when diagnosing "punches are not showing".**

PATH 1 — /iclock (the ZKTeco ADMS protocol)

Device or SBB ---> POST /iclock/cdata?SN=<serial>&table=ATTLOG
tab-delimited text, no authentication
identified only by the device serial number

PATH 2 — /api/v1 (authenticated JSON)

SBB ---> POST /api/v1/attendance/punches
JSON, X-API-Key header, per-punch acknowledgement

Both write into the same attendance_logs table.

The source column says which path a punch arrived by: "push" or "sbb".

	Path 1 — /iclock	Path 2 — /api/v1
Who can use it	Real ZKTeco hardware, and SBB in emulation mode	SBB only (hardware cannot send a credential)
Authentication	None — serial number only	Scoped API key, expirable, revocable
Acknowledgement	"OK: n" — a count	A verdict per punch: accepted / duplicate / pending / rejected
Duplicate protection	Application-level	Database unique index

	Path 1 — /iclock	Path 2 — /api/v1
Set-up effort	Point the device at host:port	Issue a key in Settings, paste it into SBB
Use it when	The device talks directly to SmartPRS	SBB is in the middle — prefer this

1.3 What happens to a punch that arrives

1. The device or bridge sends the punch with a **device code** (the PIN entered on the device, sometimes with a prefix).
2. SmartPRS looks for an employee whose emp_code or Biometric ID matches — **within that device's customer only**.
3. **Matched** → written to attendance, visible immediately in Attendance.
4. **Not matched** → held in the quarantine (attendance_pending) and listed under Settings → Pending Punches. It is not lost.
5. When an admin maps that code to an employee, every held punch for it is applied automatically, **backdated to when it actually happened**.

Why this matters on day one

At go-live nobody is mapped yet, so most of the first day's punches will land in Pending Punches. **That is correct behaviour, not a fault.** Map the codes and the attendance appears, with its original times. Do not tell the customer the punches are lost.

2. Prerequisites — what, and why

2.1 Server

Item	Minimum	Recommended	Why
CPU	2 cores	4 cores	Payroll generation and PDF export are CPU-bound
RAM	4 GB	8 GB	PHP-FPM/IIS worker pool + MySQL buffer
Disk	40 GB	100 GB SSD	Documents, payslip PDFs, backups grow steadily
OS	Windows Server 2019 / Ubuntu 22.04	Windows Server 2022 / Ubuntu 24.04	Both are supported and tested
Static IP	Yes	Yes	Devices are configured to point at a fixed address
Time	NTP synced	NTP synced	Critical. Attendance is timestamps. A drifting clock silently corrupts payroll

2.2 PHP — version and extensions

PHP 8.3. The encoded client build also needs the **SourceGuardian loader**.

Extension	Why SmartPRS needs it
openssl	HTTPS calls, licence signature verification, encrypted config values
pdo_mysql	All database access
mbstring	Multi-byte names and Indian language text
tokenizer, ctype, json	Laravel framework core
bcmath	Payroll and statutory maths — must be exact, never floating point
curl	Licence activation, update checks, WhatsApp/SMS gateways
fileinfo	Document upload type checking
dom	PDF generation and Excel import
gd	ID cards, logos, payslip images
zip	Excel import/export, backup archives

Check them before anything else

`php -m` lists the loaded extensions. A missing one usually shows up much later as a confusing error — for example a missing `zip` surfaces as "employee import does nothing", not as a clear message.

2.3 Database

- **MySQL 8 or MariaDB 10.6+.**

- Collation `utf8mb4_unicode_ci`. **Do not** use `utf8mb4_0900_ai_ci` unless the server is definitely MySQL 8 — it does not exist on MariaDB and the install will fail part-way.
- A dedicated database and a dedicated user. Do not install into an existing shared database.
- The user needs CREATE, ALTER, INDEX, SELECT, INSERT, UPDATE, DELETE. Migrations create and alter tables, so read-only or table-limited grants will fail.

2.4 Web server

Platform	Requirement	Why
IIS (Windows)	URL Rewrite module — install it before you start	Without it every URL except the home page returns 404. This is the most common Windows install failure
IIS (Windows)	PHP via FastCGI, and the app pool identity must have write access to <code>storage</code> and <code>bootstrap/cache</code>	Laravel writes compiled views, sessions and logs at runtime
nginx / Apache	PHP-FPM 8.3; <code>mod_rewrite</code> if Apache	Same pretty-URL requirement as IIS
Both	Site root points at the <code>`public`</code> sub-folder, never the application root	Pointing at the root exposes <code>.env</code> , <code>vendor</code> and <code>storage</code> to the internet

The single most important web-server rule

The document root is ``public/``. If the site root is the application folder instead, anyone can download `.env` — which contains your database password and licence secret. Check this on every install.

2.5 Network and ports

Port / direction	Purpose	Needed when
80 / 443 inbound	Users and devices reach SmartPRS	Always
3306 local	Application to MySQL	Always (localhost only — do not expose)
443 outbound	Licence activation and update checks to <code>smartprs.com</code>	On-prem, at activation and update time
587 / 465 outbound	SMTP for notification email	If email is enabled
4370 / 8000 / 8080	Device-specific ports on the customer LAN	Depends on device model — confirm with the device vendor

2.6 What to collect from the customer BEFORE you travel

#	Item	Why you need it	Got it
1	Server access — RDP/SSH, admin rights	You cannot install without it	
2	MySQL root or a user that can create databases	To create the SmartPRS database	

#	Item	Why you need it	Got it
3	The hostname or IP users will type	Goes into APP_URL; wrong value breaks links and API addresses	
4	Whether HTTPS is required, and who supplies the certificate	Affects APP_URL and API-key safety	
5	Device make, model, serial number(s)	Serial identifies the device to SmartPRS	
6	Device clock and timezone setting	A wrong device clock is silent payroll corruption	
7	Employee list with employee codes	Needed to map device PINs	
8	The biometric PIN assigned to each employee	Without this you cannot complete mapping on the day	
9	SMTP details, if email is wanted	Notifications and payslip delivery	
10	A licence file (.lic) issued for THIS machine	Each machine needs its own — see section 6	

3. Installing on Windows with IIS and MySQL

Times assume a prepared server. Budget half a day for a first install at a new customer.

3.1 Step 1 — install MySQL

1. Install MySQL 8 (or MariaDB). Choose a strong root password and record it in your handover notes.
2. Create the database and a dedicated user:

```
CREATE DATABASE smartprs
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

CREATE USER 'smartprs'@'localhost' IDENTIFIED BY '<strong-password>';
GRANT ALL PRIVILEGES ON smartprs.* TO 'smartprs'@'localhost';
FLUSH PRIVILEGES;
```

Why a dedicated user: if the application is ever compromised, the blast radius is one database rather than the whole server.

3.2 Step 2 — install PHP 8.3

1. Download the **NTS (Non Thread Safe)** x64 build — that is the one IIS FastCGI uses.
2. Extract to C:\php83.
3. Copy php.ini-production to php.ini and enable the extensions from 2.2 by removing the leading semicolon.
4. Set these values — each one matters:

```
extension_dir = "C:\php83\ext"

memory_limit = 512M          ; payroll generation for large headcounts
max_execution_time = 300      ; long payroll runs and Excel imports
upload_max_filesize = 20M     ; employee documents and the .lic file
post_max_size = 24M          ; must exceed upload_max_filesize
file_uploads = On            ; REQUIRED - see troubleshooting T-10
upload_tmp_dir = "C:\php83\tmp" ; the folder must EXIST and be writable
date.timezone = "Asia/Kolkata"
```

5. Add C:\php83 to the system PATH, open a NEW terminal and confirm:

```
php -v          # expect PHP 8.3.x
php -m          # check the 2.2 list
```

file_uploads and upload_tmp_dir

These two look harmless and are not. If file_uploads is Off, or upload_tmp_dir points at a folder that does not exist or is not writable by the app pool, the licence file upload silently arrives **empty** — and SmartPRS reports **"does not look like a License Code"**. That message has cost a full site visit before. Set them now.

3.3 Step 3 — install IIS and the URL Rewrite module

1. Server Manager → Add Roles → **Web Server (IIS)**. Include **CGI** under Application Development.

2. Install the **URL Rewrite** module from iis.net. **Do not skip this** — without it only the home page works.
3. IIS Manager → server node → **Handler Mappings** → Add Module Mapping:

```
Request path : *.php
Module       : FastCgiModule
Executable   : C:\php83\php-cgi.exe
Name         : PHP_via_FastCGI
```

3.4 Step 4 — copy the package and point the site at public/

1. Copy the client package folder to e.g. C:\inetpub\smartprs.
2. IIS Manager → Add Website:

```
Site name      : SmartPRS
Physical path   : C:\inetpub\smartprs\public    <-- the public sub-folder
Binding        : http (or https) on the port the customer will use
```

3. Confirm web.config exists in public\. It ships with the package and provides the URL rewriting Laravel needs. Its rules also hide .env, storage and vendor from being served as static files.

3.5 Step 5 — file permissions

The IIS app pool identity must be able to write two folders. Laravel compiles views, stores sessions and writes logs at runtime; without this the site shows a blank page or a 500.

```
icacls "C:\inetpub\smartprs\storage" /grant "IIS AppPool\SmartPRS):(OI)(CI)M /T
icacls "C:\inetpub\smartprs\bootstrap\cache" /grant "IIS AppPool\SmartPRS):(OI)(CI)M /T
```

Replace SmartPRS with the actual app pool name if you used a different one.

3.6 Step 6 — configure .env and run the installer

1. Copy .env.example to .env if the installer has not already done it.
2. Edit .env — at minimum DB_DATABASE, DB_USERNAME, DB_PASSWORD, and APP_URL. See the full reference in Appendix A.
3. Run the shipped installer from a terminal in the application folder:

```
INSTALL-Windows.bat
```

It checks PHP is on the PATH, creates the writable folder skeleton that zip files drop, then runs:

```
php artisan key:generate --force    # unique encryption key for this install
php artisan migrate --force         # creates all tables
php artisan db:seed --force         # reference data
php artisan optimize:clear          # clears any stale cached config/routes
```

If it prints "Nothing to migrate" on a fresh database

That is **not** success. It almost always means DB_DATABASE in .env points at a different database from the one you created — so Laravel is reading a database that already has the tables. Stop and check .env before going further. See T-01.

4. Installing on Linux with MySQL

Shown for Ubuntu 22.04/24.04 with nginx and PHP-FPM. Apache works equally well — the only difference is the virtual host and `mod_rewrite`.

4.1 Step 1 — packages

```
sudo apt update
sudo apt install -y nginx mysql-server \
    php8.3-fpm php8.3-mysql php8.3-mbstring php8.3-xml php8.3-curl \
    php8.3-bcmath php8.3-gd php8.3-zip php8.3-tokenizer unzip

php -v && php -m          # confirm 8.3 and the extension list from 2.2
```

4.2 Step 2 — database

```
sudo mysql -u root -p

CREATE DATABASE smartprs CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'smartprs'@'localhost' IDENTIFIED BY '<strong-password>';
GRANT ALL PRIVILEGES ON smartprs.* TO 'smartprs'@'localhost';
FLUSH PRIVILEGES;
```

4.3 Step 3 — deploy the package

```
sudo mkdir -p /var/www/smartprs
# copy the client package folder here, then:
cd /var/www/smartprs

sudo chown -R www-data:www-data /var/www/smartprs
sudo chmod -R 755 /var/www/smartprs
sudo chmod -R 775 storage bootstrap/cache
```

Why 775 on those two: the web server user must write compiled views, sessions, cache and logs. Everything else stays read-only, which is what you want.

4.4 Step 4 — nginx virtual host

```
server {
    listen 80;
    server_name smartprs.customer.local;
    root /var/www/smartprs/public;          # the public folder, not the app root

    index index.php;
    charset utf-8;
    client_max_body_size 24M;              # must match post_max_size

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ /\.php$ {
```

```
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
        fastcgi_index index.php;
        include fastcgi.conf;
        fastcgi_read_timeout 300;          # long payroll runs
    }

    location ~ /\.(!well-known).* { deny all; } # never serve .env
}

sudo ln -s /etc/nginx/sites-available/smartprs /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

4.5 Step 5 — php.ini

```
sudo nano /etc/php/8.3/fpm/php.ini

memory_limit = 512M
max_execution_time = 300
upload_max_filesize = 20M
post_max_size = 24M
file_uploads = On
date.timezone = Asia/Kolkata

sudo systemctl restart php8.3-fpm
```

4.6 Step 6 — run the installer

```
cd /var/www/smartprs
cp .env.example .env          # if not already present
nano .env                     # set DB_* and APP_URL

bash INSTALL-Linux.sh
```

It performs the same steps as the Windows installer: creates the writable skeleton, generates the key, migrates, seeds and clears caches.

5. First-run setup (both platforms)

5.1 The Setup Wizard — /install

1. Open the server URL in a browser. You are taken to the **Setup Wizard** at /install.
2. It runs pre-flight checks — PHP version, extensions, database connectivity, the SourceGuardian loader on encoded builds. Fix anything it flags before continuing.
3. Create the first admin login. **There is no default password** — you choose it here. Record it in the handover.
4. The wizard disables itself automatically once a user exists, so /install cannot be used to create a second admin later.

5.2 Licence activation — /activate

On-premises installs (editions **I1** / **I2** / **I3**) are licensed per machine and will not open a single protected screen until activated. Do this immediately after the wizard: sign in, go to /activate, send Ametecs the fingerprint shown there, and enter the code or file they issue.

Licensing has its own section — see section 6. It covers the three credential types, what the fingerprint is made of, offline/air-gapped activation, renewal, and every error message decoded. Read it before your first on-prem install.

5.3 Confirm the install is healthy

Check	How	Expected
Site loads	Open the URL	Login page, styled correctly
Pretty URLs work	Open /login directly	Loads — a 404 means URL Rewrite is missing (T-02)
.env is not public	Open <url>/ .env	404 or 403. If it downloads, STOP — the site root is wrong
Database	Sign in, open Employees	Page loads without error
Writable storage	Check storage/logs/laravel.log exists	File present and growing
Timezone	Settings, or php artisan tinker → config('app.timezone')	Asia/Kolkata

5.4 Queue worker — required

Notification email, late-arrival alerts and other background work run on a queue. **Without a worker these silently never send.**

Linux (systemd)

```
sudo nano /etc/systemd/system/smartprs-queue.service
```

```
[Unit]
Description=SmartPRS queue worker
After=network.target mysql.service
```

```
[Service]
User=www-data
Restart=always
RestartSec=5
WorkingDirectory=/var/www/smartprs
ExecStart=/usr/bin/php artisan queue:work --queue=mail,default --sleep=3 --tries=3

[Install]
WantedBy=multi-user.target

sudo systemctl enable --now smartprs-queue
sudo systemctl status smartprs-queue
```

Windows

Create a Scheduled Task that runs at startup, restarts on failure, and executes:

```
C:\php83\php.exe C:\inetpub\smartprs\artisan queue:work --queue=mail,default --tries=3
```

After every deployment

Run `php artisan queue:restart`. Workers hold the old code in memory and will keep running it until restarted — a classic "I deployed the fix but nothing changed" cause.

5.5 Scheduler — required

Nightly jobs (greetings, absence alerts, probation notices, subscription renewals) run from the Laravel scheduler.

Linux

```
sudo crontab -e -u www-data

* * * * * cd /var/www/smartprs && php artisan schedule:run >> /dev/null 2>&1
```

Windows

Scheduled Task, trigger **every 1 minute**, action:

```
C:\php83\php.exe C:\inetpub\smartprs\artisan schedule:run
```

It runs every minute by design — Laravel decides internally which jobs are actually due.

5.6 Backups

- Database dump nightly, retained at least 30 days. Payroll data is not reproducible.
- Also back up storage/app — employee documents and generated payslips live there.
- `.env` holds the encryption key. **Restoring a database without the matching `.env` makes encrypted fields unreadable.** Back them up together.

```
mysqldump -u smartprs -p smartprs > /backup/smartprs_$(date +%F).sql
```

6. Licensing and activation

Every on-premises install is licensed. Get this wrong and the customer cannot log in at all, so it is worth understanding rather than following blindly.

6.1 What is licensed

Edition	SMARTPRS_EDITION	What it includes	Licence gate
SaaS (Ametecs cloud)	saas	Full app + the SaaS Platform module	Not gated — subscription is handled centrally
On-prem Core HR	l1	Core HR	Gated
On-prem Advanced	l2	Core + the nine L2 modules	Gated
On-prem DNA	l3	Everything except the SaaS Platform module	Gated

The edition is set once in `.env` and read through `App\Services\Edition`. It is **compared exactly** at activation: an L2 licence will not activate on an L3 install and vice versa. Confirm the edition before asking for a licence.

6.2 How the gate behaves

`App\Http\Middleware\LicenseGate` runs on every protected request. When the licence is **missing or expired** it signs the user out and returns them to `/login`, where an admin can enter a License Code. Three paths stay reachable while blocked so the install is never bricked:

```
app/activate      — the activation screen
logout
app/change-password
```

- Expiry blocks **mid-session**, not only at login — an expired licence stops the app the moment enforcement notices.
- API and AJAX requests get 403 with a message rather than a redirect.
- **Fail-soft by design:** if the licence check itself throws, the request is let through. A licensing bug must never take a customer's HR system down. The side effect is that a badly configured install can look "unlicensed but working" — see 6.6.
- `SMARTPRS_LICENCE_ENFORCE=false` disables the gate entirely. Use it on **demo and internal installs only**, never on a paying customer's server.

6.3 The three credential types

Ametecs can issue three different things. They look different, they are verified differently, and the failure messages differ. Identify which one you are holding before troubleshooting.

Type	Looks like	Verified how	Needs internet?
Online License Code	SPRS-7K2M-9XQ4-PT3W-8NJB	Sent to the Ametecs update server, which binds this machine's fingerprint and returns a certificate	Yes — outbound 443 at activation

Type	Looks like	Verified how	Needs internet?
Offline License Code	SPRSX1.<payload>.<signature>	HMAC-SHA256 signature checked locally against SMARTPRS_LICENCE_SECRET	No
Licence file (.lic)	Two long base64 blocks joined by a dot	RSA signature checked locally against the public key built into the app	No

Online activation is the default — and that is deliberate

SMARTPRS_OFFLINE_LIC ships as **false**. With it false, a .lic is routed through the online server flow so that **the server captures the machine fingerprint** at first activation. That record is what lets Ametecs later prove which machine a licence belongs to, release it, or re-issue it.

Set SMARTPRS_OFFLINE_LIC=**true** **only for genuinely air-gapped sites**. If you set it because the site's internet was merely slow that day, you have quietly given up the server-side binding record.

6.4 The machine fingerprint — what it actually is

The fingerprint shown on /activate is not a serial number Ametecs assigns. The install computes it:

```
fingerprint = first 40 hex chars of sha256( "SMARTPRS|" + raw machine id )
```

raw machine id comes from, in order:

```
Windows  wmic csproduct get uuid
          -> fallback: powershell Win32_ComputerSystemProduct.UUID
Linux    /etc/machine-id    -> /var/lib/dbus/machine-id
macOS    IOPlatformUUID
fallback hostname | OS | architecture    <-- WEAK, see the warning below
```

Result is cached in storage/app/.machine_fp so it is computed once and stays stable for the life of the install.

The single most common cause of a "wrong machine" licence

If the hardware UUID cannot be read, the code falls back to **hostname + OS + architecture**. That fallback produces a **different** fingerprint from the real one — so a perfectly valid licence is suddenly rejected as belonging to another computer.

It happens when shell_exec is disabled in php.ini, when wmic has been removed (newer Windows Server builds) and PowerShell is not on the service account's PATH, or on a container with no /etc/machine-id.

Check the fingerprint on `activate` before requesting a licence, and check it again after any PHP configuration change. If it moved, fix the cause — do not ask for a re-issue against a fallback fingerprint.

Offline License Codes can additionally be locked to **hardware identifiers** — network MAC addresses, the chassis UUID and the BIOS serial on Windows; MACs, product_uuid, product_serial and machine-id on Linux. When present, every one of those IDs must be found on the machine.

6.5 Activating — the normal, online path

1. Complete the Setup Wizard first. Activation requires a signed-in admin.
2. Go to /activate. Record the **fingerprint** and the **edition** shown.

3. Send both to Ametecs with the customer name. Ametecs issues the code from the `/super` console.
4. Paste the code and submit. The install calls the Ametecs update server over HTTPS, which validates the code, binds the fingerprint and returns a certificate.
5. The certificate is stored locally in the `settings` table (`tenant_id = 0`, `key = licence`) as JSON: the encrypted key, the certificate, and the activation timestamps.
6. Confirm: sign out, sign back in, and open any protected screen. No redirect to `/login` means the gate is satisfied.

The stored licence key is encrypted with APP_KEY

Regenerating APP_KEY after activation makes the stored key unreadable, so a silent re-check can no longer renew itself and the install must be re-activated. **Never run `key:generate` on a live install.** It also makes every other encrypted field unreadable, so this is a wider rule than licensing.

6.6 Activating without internet

Only for air-gapped sites. Two variants, and they fail differently.

Offline License Code — SPRSX1...

- SMARTPRS_LICENCE_SECRET in the client `.env` must be **byte-identical** to the value on the Ametecs licensing server. The signature is an HMAC keyed on it.
- The secret has **no working default in production**. If it is unset, the app refuses to verify anything — and because the gate is fail-soft, requests pass through instead of showing a clear error. An install that seems to ignore its licence entirely is usually this. Set the secret.
- The code may carry an **account-email lock**; then SMARTPRS_LICENCE_EMAIL on the install must match it exactly.

Licence file — `license.lic`

- Lives in the application root unless SMARTPRS_LICENSE_FILE points elsewhere. Upload it on `/activate` or place it on disk.
- It **must carry a fingerprint for this machine**. A "floating" `.lic` with no fingerprint is refused outright — one licence, one machine is enforced, not merely intended.
- It is **re-verified on disk at every check**, not just at activation. Deleting the file, or copying it to a second server, blocks access even though a certificate is stored in the database.

6.7 Expiry, renewal and offline grace

Behaviour	What happens
Offline grace	Once activated, the install keeps working without internet until the stored expiry date passes. There is no daily phone-home requirement
Expiry mode renew	On expiry the install is blocked until a new code is entered. This is the mode for every offline code — there is no server to pull a renewal from
Expiry mode notify	Online installs only. On expiry the install silently re-checks the stored key against the server; if Ametecs has renewed it server-side, the install unblocks itself with no site visit
Revocation	A code revoked while the machine was online is remembered locally. Re-entering it stays blocked even after the machine goes offline

Behaviour	What happens
Seat limit	When the certificate carries a device/seat limit, it is compared against active on-roll employees — exited, inactive, resigned, terminated, left, absconded, archived and deleted staff do not count. Tell the customer this before they argue about the number

6.8 Hardware changes, re-imaging and migration

- Changing the motherboard, re-imaging the server, or moving to a new VM changes the machine UUID, therefore the fingerprint, therefore the licence stops working. **This is expected.**
- Plan it: ask Ametecs to release the old binding in /super **first**, then activate on the new machine. Doing it the other way round can trip the device limit.
- Deleting storage/app/.machine_fp is harmless in itself — it is recomputed. It is only a problem if the underlying UUID has since become unreadable (6.4).
- Restoring a database backup onto a different machine carries the old certificate with it. It will fail the fingerprint check and need re-activation. Budget for that in any DR test.

6.9 Activation error messages, decoded

These are the messages the customer will read out to you over the phone.

Message the user sees	What it really means	What to do
"locked to a different computer"	The .lic fingerprint does not match this machine	Confirm the fingerprint on /activate (check 6.4 first — it may have drifted), then have it re-issued
"not tied to a computer"	The .lic carries no fingerprint at all	Send Ametecs this machine's fingerprint; a floating licence is refused by design
"invalid or has been tampered with"	RSA or HMAC signature check failed — wrong keypair, wrong secret, or the code was re-typed with an error	Copy-paste the exact code. If it still fails, the build and the licensing server keys do not match — escalate
"does not look like a SmartPRS licence file"	The submitted text is not a recognised format — very often an empty upload	Check file_uploads=0n and that upload_tmp_dir exists and is writable. See T-10
"cannot read its licence key"	The public key inside this build did not load	A broken build — escalate to Ametecs
"is for SmartPRS-L2, but this installation is SmartPRS-L3"	Edition mismatch, compared exactly	Re-issue for the correct edition, or correct SMARTPRS_EDITION if the install itself is wrong
"expired on YYYY-MM-DD"	The code was already past its date when entered	Obtain a current code
"locked to a different device"	A hardware-locked code, and one of the required MAC/UUID/serial values is absent	Usually a NIC change or a disabled adapter. Re-issue against the current IDs

Message the user sees	What it really means	What to do
"issued for a different account email"	The code carries an email lock and SMARTPRS_LICENCE_EMAIL does not match	Set the configured email, or have the code re-issued
"has been revoked"	Ametecs revoked it, and this install knows	Commercial matter — do not work around it

6.10 For the Ametecs build engineer, not the site engineer

- Run the client package through **SourceGuardian encoding** before shipping. The client server then needs the matching loader — this is why it is a prerequisite in 2.2.
- Set a **strong, unique** `SMARTPRS_LICENCE_SECRET` per release line, and never ship the private licence key inside the package.
- The RSA **private** key lives only on the /super server. Rotating the keypair invalidates **every** `.lic` **already issued** — every on-prem customer would need re-issuing. Treat rotation as a release event, not a maintenance task.

7. Connecting biometric devices and Smart Biometric Bridge

7.1 Choose the path first

Situation	Use	Why
Device supports Cloud Server / ADMS and points straight at SmartPRS	Path 1 — /iclock	No extra software. The device dials out, so it works behind NAT
Mixed vendors, or devices that cannot reach the server directly	SBB	The bridge collects locally and forwards
SBB is being installed anyway	Path 2 — /api/v1	Authenticated, per-punch acknowledgement, database-level duplicate protection

7.2 Path 1 — pointing a device straight at SmartPRS

1. On the device: Menu → Comm → Cloud Server / ADMS.
2. Set **Server address** to the SmartPRS host and **port** to 80 or 443. The URL path is fixed in firmware — you cannot change it, and SmartPRS already listens where the firmware expects.
3. Set the device clock and timezone. **Check this twice.** The device sends local wall-clock time and SmartPRS stores exactly what it receives.
4. Punch once on the device.
5. In SmartPRS: Settings → **Biometric Device Setup**. The device should appear automatically, identified by its serial.

On the shared Ametecs cloud only — claim the device

A device that registers itself arrives with no customer attached, and a device with no customer **matches nobody by design** (this is what stops one customer's punch landing in another's payroll). Go to Settings → **Pending Punches** → "Unassigned devices" and assign it to the right workspace. Until you do, its punches are held, not filed.

7.3 Path 2 — SBB with an API key

1. In SmartPRS: **Settings** → **API Keys**.
2. Create a key. Name it for the site, e.g. "SBB — Chennai Plant". Scope: ingest.
3. **Copy the key immediately.** It is shown once and stored only as a hash. If lost, revoke it and create another.
4. The same page shows the exact addresses to enter in SBB — base URL, both endpoints, and the header format. Copy them from there rather than typing from memory; they are generated from the server's own address and are always correct.
5. In SBB, enter the server address and the key, per SBB's own documentation.
6. Verify before sending real traffic:

```
curl -H "X-Api-Key: sk_prs_xxxx_..." https://<host>/api/v1/ping
```

A correct reply names the customer:

```
{"ok":true,"app":"SmartPRS","tenant_id":3,
  "company_name":"Acme Textiles Pvt Ltd","scopes":["ingest"],
```

```
"timezone":"Asia/Kolkata","server_time":"2026-08-18T09:41:00+05:30"}
```

Read the company name back before you send a single punch

This is the whole point of ping. If it names a different customer, the key is scoped wrongly — stop and fix it. Catching that here takes a minute; catching it after a week of punches means unpicking another customer's payroll.

7.4 Timestamp format — confirm this

Punch times must carry no timezone offset

SmartPRS expects plain local wall clock: YYYY-MM-DD HH:MM:SS. A time carrying +05:30 or Z is **rejected** on the /api/v1 path rather than guessed at.

On the older /iclock path a timestamp with an offset can be stored with the digits taken literally — a punch sent as 09:41:00+00:00 would be recorded as 09:41 when the person actually clocked in at 15:11. **Confirm the sender uses the plain format before go-live.**

And confirm the device clock matches the server timezone shown on the API Keys page.

7.5 Mapping employees to device codes

A punch only becomes attendance when SmartPRS knows which employee a device code belongs to. There are three ways to establish that:

Method	Where	Best for
Employee code equals the device PIN	Nothing to do	Sites that set the PIN to the employee code
Biometric ID field on the employee	Directory → employee profile	A handful of employees
biometric_id column in the employee CSV	Employee import	Bulk — use this at go-live
Pending Punches → Map	Settings → Pending Punches	Cleaning up whatever was missed

An emp_prefix can be set per device when the device sends a bare number but employee codes carry a prefix — device 1043 plus prefix EMP matches EMP1043.

8. Go-live verification

Complete this at every site before you leave. It is the difference between "installed" and "working".

#	Test	Expected result	Pass
1	Open the site URL	Login page loads, styled	
2	Open <url>/ .env	404 or 403 — never a download	
3	Sign in as the admin created in /install	Dashboard loads	
4	Licence status on /activate	Active, not expired, edition correct, bound to this machine	
4b	Record the fingerprint on the handover sheet	Written down — you will need it at renewal	
5	Create a test employee	Saves and appears in the Directory	
6	Punch on the device	Device shows success	
7	Punch appears in Attendance (if mapped)	Correct employee, correct time	
8	Or appears in Pending Punches (if not mapped)	Listed, NOT lost	
9	Map that code to the employee	Held punches applied, backdated	
10	Punch again	No duplicate row	
11	Queue worker running	Service/task active	
12	Scheduler running	Task present, fires every minute	
13	Backup job configured and tested once	Dump file produced	
14	Handover sheet completed	Admin login, DB details, licence, device serials	

9. Day-to-day usage

9.1 The screens that matter for attendance

Screen	Where	What it is for
Biometric Device Setup	Settings	Add, edit and test devices; map a device to a branch; sync now
Pending Punches	Settings	Punches held because their code is not mapped; unassigned devices
API Keys	Settings	Issue, list and revoke keys for SBB; shows the connection details
Employee Mapping card	Biometric Device Setup	Link unknown device IDs to employees
Daily Attendance / Attendance Report	Attendance	What actually landed
Attendance Correction	Attendance	Fix an individual punch with an audit trail

9.2 Routine: clearing Pending Punches

1. Open Settings → Pending Punches.
2. Each row is one device code on one device, with a count and first/last seen.
3. Pick the employee and press **Map**. Because the row carries the device serial, mapping is confined to that device — two devices can both use PIN 5 for different people.
4. Held punches are applied immediately, backdated to the real punch time.
5. The list should normally be empty. A growing list means new employees are being enrolled on devices without being mapped in SmartPRS.

9.3 Routine: after any code deployment

```
php artisan optimize:clear
php artisan queue:restart
# then reload the page with Ctrl+F5
```

Why all three: Laravel caches config and routes; the queue worker holds old code in memory; the browser caches the front-end. Skipping any one of them produces "the fix did not work" when in fact it did.

9.4 Health checks worth doing monthly

- Pending Punches is empty or explained.
- No unassigned devices (cloud only).
- Queue worker and scheduler still running.
- Backups are being produced **and** one has been restored somewhere as a test.
- Licence expiry date is not approaching.
- storage/logs/laravel.log is not full of repeating errors.

10. Troubleshooting

Every entry here is a fault that has occurred on a real install.

10.1 Installation and start-up

Ref	Symptom	Cause	Fix
T-01	"Nothing to migrate" on a database you just created	DB_DATABASE points at a DIFFERENT database that already has the tables	Check .env. Confirm with <code>php artisan migrate:status</code> and by listing tables in the database you expected
T-02	Home page works, every other URL 404s	IIS URL Rewrite module not installed, or Apache <code>mod_rewrite</code> off	Install URL Rewrite; confirm <code>web.config</code> is in <code>public\</code>
T-03	Blank white page, or HTTP 500 with no detail	App pool cannot write <code>storage / bootstrap\cache</code>	Apply the <code>icacls</code> grants in 3.5; check <code>storage/logs/laravel.log</code>
T-04	.env downloads in the browser	Site root points at the application folder, not <code>public</code>	Serious. Re-point the site root to <code>public</code> , then rotate the DB password and <code>APP_KEY</code>
T-05	Install fails on collation	<code>utf8mb4_0900_ai_ci</code> used on MariaDB	Set <code>DB_COLLATION=utf8mb4_unicode_ci</code> and re-run
T-06	"Class not found" or a missing vendor file	Incomplete package copy or interrupted install	<code>composer install --no-dev --optimize-autoloader</code> , or re-copy the package
T-07	Errors mentioning a missing file under vendor/	A partial vendor tree — one data file missing is enough	<code>composer reinstall <the/package></code> — e.g. <code>symfony/translation</code>

10.2 Licence and activation

Ref	Symptom	Cause	Fix
T-10	"does not look like a License Code"	The upload arrived empty — this is not a licensing fault	Check <code>file_uploads=0n</code> , <code>upload_tmp_dir</code> exists and is writable by the app pool, and the <code>.lic</code> is not 0 bytes
T-11	Licence rejected as invalid signature	The <code>.lic</code> was signed by a different keypair than this build embeds	Escalate to Ametecs — the licence server keys and the build must match
T-12	Licence rejected on a second PC	Correct — a <code>.lic</code> is bound to one machine fingerprint	Issue a licence for that machine's fingerprint
T-13	L2 licence will not activate on L3	The edition string is compared exactly	Issue a licence for the correct edition
T-14	Install shows the wrong edition after a config cache	Stale cached config	<code>php artisan config:clear</code> then <code>optimize:clear</code>

Ref	Symptom	Cause	Fix
T-15	Demo or test install blocked by the licence gate	Enforcement on where it is not wanted	Set SMARTPRS_LICENCE_ENFORCE=false on that install only
T-16	A licence that worked yesterday is now "wrong machine", hardware unchanged	The fingerprint drifted onto the weak fallback — shell_exec disabled, wmic missing, or storage/app/.machine_fp deleted while the UUID was unreadable	See 6.4. Restore the ability to read the hardware UUID, confirm the fingerprint returns to its old value, and only then ask for a re-issue
T-17	Offline SPRSX1 code rejected as tampered	SMARTPRS_LICENCE_SECRET differs from the licensing server, or is unset	Set the identical secret on both sides. Never generate a new one per install
T-18	App behaves as if unlicensed but nothing is blocked	The licence check itself is throwing and the gate is fail-soft	Check storage/logs/laravel.log. Usually a missing SMARTPRS_LICENCE_SECRET in production
T-19	Licensed install stops working after a restore or key:generate	APP_KEY changed, so the stored encrypted licence key cannot be read	Restore the original .env. If it is gone, re-activate — and note every other encrypted field is affected too

10.3 Attendance and devices

Ref	Symptom	Cause	Fix
T-20	Punches are not showing in Attendance	The code is not mapped to an employee	Settings → Pending Punches. They are held there, not lost. Map and they apply, backdated
T-21	On the cloud: a whole device stopped delivering	The device has no customer assigned, so it matches nobody by design	Settings → Pending Punches → Unassigned devices → assign it. Then map the codes
T-22	Punch times are wrong by a fixed number of hours	Device clock/timezone wrong, or the sender is adding a timezone offset	Fix the device clock. Confirm the sender uses YYYY-MM-DD HH:MM:SS with no offset (7.4)
T-23	Duplicate punches	A re-send arriving before the duplicate protection was in place	Run <code>php artisan attendance:dedupe --dry</code> first, review, then without <code>--dry</code>
T-24	Device does not appear at all	It never reached the server	Check the device Cloud Server/ADMS address and port; check the firewall; punch again to force contact
T-25	API returns 401	Key missing, wrong, revoked, expired, or not bound to a workspace	Re-check the key. Test with <code>/api/v1/ping</code> before blaming the sender

Ref	Symptom	Cause	Fix
T-26	API returns 403	The key lacks the ingest scope	Create a key with ingest
T-27	API returns a punch as rejected with TIME_FORMAT	The timestamp carried an offset or was malformed	Correct the sender to plain local wall clock. This is working as intended — it prevents wrong times entering payroll

10.4 Email, jobs and background work

Ref	Symptom	Cause	Fix
T-30	No notification emails at all	No queue worker running	Start it (5.4) and confirm it stays running
T-31	Emails stopped after a deployment	The worker is holding the old code	php artisan queue:restart
T-32	Nightly jobs never run	Scheduler not configured	Add the every-minute cron or Scheduled Task (5.5)
T-33	Site slow when email is sending	SMTP is slow and something is sending inline	Ensure the queue worker is running so sending is asynchronous

10.5 Development-machine oddities (not customer faults)

Ref	Symptom	Cause	Fix
T-40	ERR_CONNECTION_REFUSED on localhost:8000	Server not running, or started with --host=0.0.0.0 while localhost resolves to IPv6	Use http://127.0.0.1:8000, or plain php artisan serve
T-41	Console shows Undefined array key 0 in ServeCommand	Cosmetic bug in Laravel's dev-server request logger, triggered by another exception	Ignore. Cannot occur under IIS or nginx
T-42	A page 500s on a legacy screen	Old Blade views reference routes that no longer exist	Use the current screens; report the URL to Ametecs

10.6 Where to look

What	Where
Application errors	storage/logs/laravel.log — newest entries at the end
Attendance ingest activity	Same log; search for sbb.ingest.batch, sbb.punch.rejected, attendance.pending.promoted
Web server errors	IIS: Event Viewer + %SystemDrive%\inetpub\logs. nginx: /var/log/nginx/error.log
PHP-FPM	/var/log/php8.3-fpm.log
MySQL	/var/log/mysql/error.log, or the Windows data directory

What	Where
What migrated	<code>php artisan migrate:status</code>
What routes exist	<code>php artisan route:list</code>
Held punches	Settings → Pending Punches, or <code>SELECT * FROM attendance_pending WHERE resolved_at IS NULL</code>

Appendix A — .env reference

Key	Set it to	Why it matters
APP_ENV	production	Turns off debug behaviour
APP_DEBUG	false	Never true in production — error pages would expose configuration
APP_KEY	generated	Encryption key. Back it up with the database or encrypted fields become unreadable
APP_URL	The exact URL users type	Links, PDF assets and the API addresses shown on the API Keys screen
APP_TIMEZONE	Asia/Kolkata	Attendance is timestamps. Must match the device clocks
DB_DATABASE / USERNAME / PASSWORD	The dedicated database and user	Wrong database is the T-01 trap
DB_COLLATION	utf8mb4_unicode_ci	Works on MariaDB and MySQL 8 alike
CACHE_STORE / SESSION_DRIVER / QUEUE_CONNECTION	database	Never `redis` unless Redis is actually installed
SMARTPRS_DEPLOYMENT	onprem or saas	Decides whether a licence is required
SMARTPRS_EDITION	as shipped	Compared exactly against the licence
SMARTPRS_LICENCE_ENFORCE	true in production	false only for demo/test installs
SMARTPRS_LICENCE_SECRET	a long random string	Change it. The shipped template value is a placeholder. Must be identical to the licensing server for offline codes; unset in production means no code can be verified at all (T-18)
SMARTPRS_LICENCE_EMAIL	blank, or the account email	Only needed when the issued code is email-locked. Must match exactly
SMARTPRS_OFFLINE_LIC	false	Leave false so first activation goes online and the server records the fingerprint. true for air-gapped sites only
SMARTPRS_LICENSE_FILE	blank	Blank means license.lic in the application root. Set only if the file must live elsewhere
SMARTPRS_UPDATE_URL	leave as shipped	The Ametecs server used for online activation and update checks. Needs outbound 443
MAIL_MAILER	smtp when email is wanted	Left at log nothing is delivered

Appendix B — command cheat sheet

```
# after any deployment
php artisan optimize:clear
php artisan queue:restart

# what state is the schema in
php artisan migrate:status
php artisan migrate --force

# what routes exist
php artisan route:list
php artisan route:list --path=api/v1

# duplicate attendance repair – ALWAYS dry-run first
php artisan attendance:dedupe --dry
php artisan attendance:dedupe

# background work
php artisan queue:work --queue=mail,default --tries=3
php artisan schedule:run

# maintenance window
php artisan down
php artisan up

# backup
mysqldump -u smartprs -p smartprs > smartprs_$(date +%F).sql
```

Appendix C — handover sheet

Complete one per site and leave a copy with the customer.

Item	Value
Customer / site	
Server hostname / IP	
Application URL	
Application folder path	
Database name / user	
Admin login created	
Edition installed (I1 / I2 / I3)	
Licence fingerprint	
Licence type (online code / offline code / .lic file)	

Item	Value
Licence expiry	
Seat limit, if any	
Device make / model / serial(s)	
Ingest path in use (/iclock or /api/v1)	
API key prefix (if used)	
Queue worker configured	
Scheduler configured	
Backup schedule + location	
Installed by / date	
Verification (section 8) completed	

SmartPRS by Ametecs · 18 August 2026 · ejaz@ameteccsindia.com · WhatsApp 9000098877